

Çekirdek Katmandan Sınırlı Bağlamlara: Domain Odaklı Mimariye Evrimsel Geçiş

Ömer Faruk Yavuz

December 2025

Giriş

Çağdaş istemci tarafı uygulama geliştirmede iki sorun çoğu zaman iç içe geçmiş biçimde ortaya çıkar.

Birincisi, aynı iş kurallarının mobil ve web gibi istemcilerinde ayrı ayrı yazılması nedeniyle oluşan tekrar ve buna bağlı davranış sapmasıdır.

İkincisi, iş mantığı arayüz koduna gömülü kaldığı sürece, işin kendi kavramlarının kod içinde hiçbir zaman görünür hale gelmemesi ve karmaşıklık arttıkça sistemin anlaşılabilirliğini yitirmesidir.

Bu çalışma her iki soruna tek bir evrimsel hat üzerinden yanıt vermektedir. Hattın başlangıcı, iş mantığını barındıran merkezi bir *çekirdek* katman ile bu katmanı tüketen mobil ve web istemcilerinden oluşan üç bileşenli bir yapıdır. Hattın sonu ise, bu çekirdeğin domain odaklı tasarım (Domain-Driven Design) ilkelerine göre yeniden yapılandırılmış hâlidir.

Çalışmanın kapsamı, geçişin yalnızca ön koşullarını hazırlamakla sınırlı değildir; sınırlı bağlamların nasıl keşfedileceği, model yapı taşlarının nasıl seçileceği, bağlamlar arası ilişkilerin nasıl yönetileceği ve kalıcılık ile domain modeli arasındaki gerilimin nasıl çözüleceği de ele alınmaktadır. Amaç, okuyucunun mevcut bir katmanlı yapıdan başlayarak domain odaklı bir mimariye kendi başına geçebilmesidir.

Anlatım boyunca kod örneklerinden kaçınılmış, kavramlar ve karar ölçütleri düzeyinde kalınmıştır. Bunun nedeni, domain odaklı tasarımın esas zorluğunun sözdizimsel değil, kavramsal olmasıdır.

Anahtar Kavramların Açıklamaları

Bu bölümde, çalışmada geçen kavramlar tanımlanmakta ve her biri için günlük hayattan veya yazılım dışı alanlardan anlaşılır örnekler verilmektedir. Örneklerin çoğunda, tutarlılık sağlamak amacıyla bir *restoran zinciri* ve bir *kargo şirketi* benzetmesi kullanılmaktadır.

Mimari Katmanlar

Çekirdek katman (core layer)

Uygulamanın iş kurallarının toplandığı, hiçbir arayüze bağlı olmayan merkezi bölümdür.

Örnek: Bir restoran zincirinin genel merkezinde hazırlanan reçete kitabı. Kitapta yemeğin nasıl yapılacağı yazar; hangi şubede hangi tabakla servis edileceği yazmaz. Her şube aynı kitabı kullanır.

Domain katmanı (domain layer)

Çekirdeğin içinde, işin kavramlarını ve kurallarını barındıran en iç bölüm. Dış dünyadan tamamen habersizdir.

Örnek: Reçete kitabındaki “sos, kaynama noktasına gelmeden kremadan ayrılmaz” kuralı. Bu kural, mutfağın hangi markanın ocağını kullandığını bilmez ve bilmesi de gerekmez.

Uygulama katmanı (application layer)

Bir işin baştan sona hangi sırayla yürütüleceğini düzenleyen bölüm. Kendisi karar vermez, kararları sıraya koyar.

Örnek: Mutfaktaki şef garson. Kendisi yemek pişirmez; siparişi alır, malzemeyi depodan ister, aşçıya iletir, tabak hazır olunca servise haber verir.

İstemci katmanı (client layer)

Kullanıcının doğrudan gördüğü ve etkileşime girdiği bölüm.

Örnek: Restoranın salonu ve menüsü. Aynı yemek, havaalanı şubesinde karton kutuda, şehir merkezinde porselen tabakta sunulur; yemeğin kendisi değişmez.

Hazırlık mimarisi (preparatory architecture)

Henüz tam bir domain modeli kurulmamışken, ileride kurulabilmesi için zemin hazırlayan ara yapı.

Örnek: Bir eve tadilat öncesi yapılan boşaltma işlemi. Duvar örülmemiştir, ama örülebilmesi için ortam açılmıştır.

Bağımlılık yönü (dependency direction)

Hangi bölümün hangisini tanıdığını belirleyen kural: dış bölümler içi tanır, iç bölümler dışı tanımaz.

Örnek: Anayasa ile yönetmelik ilişkisi. Yönetmelik anayasaya atıf yapar; anayasa hiçbir yönetmelikten haberdar değildir. Yönetmelik değişince anayasa değişmez.

Bağımlılığın tersine çevrilmesi (dependency inversion)

İç katmanın ihtiyacını kendi diliyle bir talep olarak tanımlaması, bu talebin nasıl karşılanacağını ise dış katmana bırakması.

Örnek: Bir aşçının “bana 200 gram taze fesleğen lazım” demesi. Fesleğenin hangi tedarikçiden, hangi araçla geleceği satın alma biriminin işidir; aşçı bunu bilmez.

Dil ve Anlam

Ortak dil (ubiquitous language)

İşi bilen kişi ile yazılımı yapan kişinin aynı kavramı aynı kelimeyle ifade etmesi.

Örnek: Kargo şirketinde herkesin “teslimat denemesi” demesi. Biri “dağıtım girişimi”, diğeri “kapı çalma” derse, aynı şeyden bahsedip bahsetmedikleri hiçbir zaman netleşmez.

Terim sözlüğü (glossary)

Ortak dilin yazıya dökülmüş hâli; her terimin tanımı ve hangi durumda geçerli olduğu.

Örnek: Bir hastanenin resmî kısaltma listesi. “Yatan hasta” ile “gözlem hastası” arasındaki farkın yazılı olması, iki farklı birimin aynı kişiyi farklı sayması sorununu ortadan kaldırır.

Dil kırılması (language breakdown)

Aynı kelimenin işin farklı yerlerinde farklı anlama gelmeye başlaması.

Örnek: Kargo şirketinde “teslim edildi” ifadesi. Dağıtım biriminde “paket alıcıya verildi” anlamına gelir; muhasebe biriminde “ödeme tahsil edildi” anlamına gelir. Aynı kelime, iki farklı gerçekliği anlatmaktadır.

Terim çatışması (term conflict)

Dil kırılmasının fark edilmiş hâli. Bir sorun değil, sınır keşfinin en değerli bulgusudur.

Örnek: Yukarıdaki “teslim edildi” durumunun toplantıda ortaya çıkması. Doğru tepki, tek bir tanımda uzlaşmaya çalışmak değil, iki ayrı alanın varlığını kabul etmektir.

Domain uzmanı (domain expert)

İşin kendisini bilen, ancak yazılım geliştirmeyen kişi.

Örnek: Otuz yıllık dağıtım şefi. Hangi durumda paketin şubeye geri döneceğini, hangi durumda komşuya bırakılabileceğini kimsenin yazmadığı biçimde bilir.

Sınırlar ve Bağlamlar

Sınırlı bağlam (bounded context)

Belirli bir modelin ve terimlerinin geçerli olduğu alan. İçeride her terimin tek anlamı vardır.

Örnek: Bir hastanede “hasta” kavramı. Poliklinikte randevusu ve şikâyeti olan kişidir; muhasebede sigorta numarası ve fatura bakiyesi olan kişidir. İki ayrı alan, iki ayrı “hasta” modeli.

Bağlam haritası (context map)

Sistemdeki bütün bağlamların ve aralarındaki ilişkilerin gösterildiği üst düzey resim.

Örnek: Bir havalimanının birimler arası akış şeması. Check-in, güvenlik, bagaj, uçuş operasyonu birimlerinin hangisinin hangisine bilgi geçirdiğini gösterir.

Yukarı akış / aşağı akış (upstream / downstream)

İlişkinin hangi tarafının değişikliği dayattığını, hangi tarafının uyum sağlamak zorunda olduğunu belirten konum.

Örnek: Merkez bankası ile ticari bankalar. Merkez bankası faiz kararını kendi gerekçesiyle alır (yukarı akış); ticari bankalar bu karara uyum sağlamak zorundadır (aşağı akış).

Ortak çekirdek (shared kernel)

İki bağlamanın modelin bir bölümünü ortaklaşa sahiplenmesi. Değişiklik iki tarafın da onayını gerektirir.

Örnek: İki komşunun ortak kullandığı bahçe duvarı. Hiçbiri diğerine sormadan duvarı yıkamaz veya yükseltmez.

Müşteri–tedarikçi (customer–supplier)

Aşağı akış tarafın ihtiyaçlarının, yukarı akış tarafın planına girebildiği ilişki.

Örnek: Restoran ile düzenli çalıştığı sebze tedarikçisi. Restoran “önümüzdeki ay menüde enginar olacak” dediğinde tedarikçi buna göre hazırlık yapar.

Uyum sađlayan (conformist)

Aşğı akış tarafın, karşı tarafın modelini olduğı gibi kabul etmesi.

Örnek: Küçük bir işletmenin vergi dairesinin beyan formatını aynen kullanması. Pazarlık gücü yoktur; formatı tartışmak yerine ona uyar.

Yozlaşma önleyici katman (anti-corruption layer)

Dış dünyanın modelini kendi diline çeviren ara katman.

Örnek: Yabancı bir heyetle yapılan görüşmedeki çevirmen. Karşı tarafın kavramları doğrudan salona girmez; çevirmenden geçerek girer. Karşı taraf terminolojisini değıştirse bile, salondaki dil bozulmaz.

Yayımlanmış dil (published language)

Tarafların hiçbirinin iç modeline ait olmayan, açıkça tanımlanmış ortak iletişim biçimi.

Örnek: Uluslararası havacılıkta kullanılan standart uçuş bildirim formatı. Ne Türk ne Alman hava trafik sisteminin iç yapısıdır; ikisinin de kabul ettiğı ortak bir sözleşmedir.

Ayrı yollar (separate ways)

İki bağlam arasında bilinçli olarak hiç entegrasyon kurmamak.

Örnek: Bir şirketin yemek kartı sistemiyle proje takip sistemini birbirine bağlamaması. Bağlamanın maliyeti, sağlayacağı faydayı aşar.

Değışim birlikteliğı (change coupling)

Hangi kuralların birlikte değışme eğiliminde olduğunun gözlemlenmesi. Sınır keşfinde ölçülebilir bir göstergedir.

Örnek: Bir mağazada indirim politikası her değıştiğinde etiket basımının da değışmesi, ama stok sayımının hiç etkilenmemesi. İlk ikisi aynı alana, üçüncüsü ayrı bir alana aittir.

Model Yapı Taşları

Kimlikli nesne (entity)

Nitelikleri değışse bile aynı kalan, sürekliliğı olan kavram.

Örnek: Bir kişi. Adını değıştirebilir, taşınabilir, kilo alabilir; yine aynı kişidir. Eşitlik, özelliklerine değıl kimliğine bakılarak belirlenir.

Değer nesnesi (value object)

Yalnızca taşıdığı değerle tanımlanan, kimliği olmayan kavram.

Örnek: Yüz lira. “Hangi yüz lira” diye sorulmaz; iki yüz liralık aynı şeydir. Cüzdandaki banknotun seri numarası kimseyi ilgilendirmez.

Bütünlük birimi (aggregate)

Birlikte tutarlı kalması gereken nesnelerin oluşturduğu, dışarıya tek kapıdan açılan küme.

Örnek: Bir sipariş ve satırları. Sipariş toplamı ile satırların toplamı asla birbirini tutmayan bir hâlde bulunamaz. Bu nedenle ikisi tek bir bütün olarak ele alınır.

Bütünlük birimi kökü (aggregate root)

Kümenin dışarıya açılan tek girişi; kuralların uygulanmasından sorumlu nesne.

Örnek: Siparişin kendisi. Bir satırı silmek isteyen, satıra doğrudan uzanmaz; siparişe “şu satırı çıkar” der. Sipariş de bu sırada toplamı yeniden hesaplar.

Domain olayı (domain event)

İşin akışında anlam taşıyan, gerçekleşmiş bir durumu bildiren kavram. Geçmiş zamanda adlandırılır ve değişmez.

Örnek: “Kargo teslim edildi.” Bu olay geri alınamaz. Yanlışlık varsa “iade başlatıldı” adlı ikinci bir olayla telafi edilir.

Domain servisi (domain service)

Hiçbir tek nesneye doğal olarak ait olmayan, birden fazla nesneyi ilgilendiren iş kuralı.

Örnek: İki hesap arasında para transferi. Ne gönderen hesabın ne alan hesabın tek başına sorumluluğudur; ikisini birden ilgilendiren ayrı bir işlemdir.

Depo (repository)

Bütünlük birimlerinin saklanması ve geri getirilmesini işin diliyle ifade eden soyutlama.

Örnek: Kütüphane görevlisi. “Şu yazarın son kitabını istiyorum” denir; görevlinin kitabı hangi rafta, hangi depoda bulunduğu istekte bulunam ilgilendirmez.

Üretim noktası (factory)

Bir nesnenin oluşturulması sırasında sağlanması gereken kuralları tek yerde uygulayan sorumluluk.

Örnek: Nüfus müdürlüğü. Kimlik belgesi rastgele üretilemez; belirli kontrollerden geçmiş bir başvuru sonucunda tek bir noktadan çıkar. Böylece geçersiz bir kimlik hiçbir zaman var olmaz.

Kullanım senaryosu (use case)

Kullanıcının bakış açısından anlamlı, baştan sona bir iş akışı.

Örnek: “Randevu iptal et.” İçinde birden çok adım vardır (randevuyu bul, iptal edilebilir mi kontrol et, iptal et, bilgilendirme gönder), ama dışarıdan tek bir eylemdir.

Aktarım nesnesi (data transfer object)

Yalnızca veri taşımak amacıyla oluşturulan, hiçbir kural içermeyen sade yapı.

Örnek: Bir kurumun gönderdiği özet bilgilendirme yazısı. Kurumun iç kayıtlarının kendisi değildir; dışarıya gösterilmek üzere hazırlanmış sade bir özettir.

Tutarlılık ve Kalıcılık

Değişmez (invariant)

Her koşulda geçerli olması gereken, ihlal edilmesine hiçbir an izin verilmeyen kural.

Örnek: “Bir uçuşta satılan bilet sayısı, koltuk sayısını aşamaz.” Bu kural bir an bile ihlal edilemez; ihlal edildiği anda sistem anlamını yitirir.

Anlık tutarlılık (immediate consistency)

İlgili tüm verilerin aynı anda doğru olmasının zorunlu olduğu durum.

Örnek: Banka havalesinde gönderenden düşülen tutarla alıcıya eklenen tutar. İki arasında bir saniyelik bile tutarsızlık kabul edilemez.

Gecikmeli tutarlılık (eventual consistency)

Verilerin bir süre farklı olabildiği, ancak sonunda uyuşacağının bilindiği durum.

Örnek: Kargo teslim edildikten sonra müşteriye bilgilendirme mesajının birkaç dakika gecikmeyle gitmesi. Bu gecikme kimseyi zarara uğratmaz; sonunda bilgi yerine ulaşır.

İşlem sınırı (transaction boundary)

Ya tamamı gerçekleşecek ya da hiçbiri gerçekleşmeyecek değişikliklerin kapsamı.

Örnek: Nikâh töreni. İki taraf da evli sayılır ya da hiçbiri sayılmaz. “Biri evlendi, diğlerinin işlemi yarım kaldı” diye bir durum yoktur.

Şema baskısı (schema pressure)

Veri saklama biçiminin, iş modelinin şeklini bozmaya başlaması.

Örnek: Bir arşiv dolabının çekmece boyutuna göre belgelerin katlanması. Zamanla belgelerin içeriği değil, dolaba sığma kaygısı belirleyici hâle gelir.

Okuma–yazma ayrımı (read–write separation)

Veri deęiřtiren iřlemlerle yalnızca görüntüleme yapan iřlemlerin farklı disiplinlere tabi tutulması.

Örnek: Bir bankada giře ile bakiye ekranı farkı. Para çekmek için kimlik, imza ve onay gerekir; bakiyeyi görüntülemek için bunların hiçbirini gerekmez.

Deęiřmezlik (immutability)

Bir nesnenin oluřturulduktan sonra deęiřtirilmemesi; deęiřiklik gerektiğinde yenisinin üretilmesi.

Örnek: Bir fatura. Yanlıř kesilen fatura üzerinde düzeltme yapılmaz; iptal edilip yenisi kesilir. Eski belge olduęu gibi durur.

Model Kalitesi

Anemik model (anemic model)

Nesnelerin yalnızca veri taşıdıęı, tüm kuralların dışarıdaki iřlevlerde biriktięi yapı.

Örnek: İçi boş bir dosya klasörü ile ayrı bir talimat kitapçıęı. Klasör kendi başına hiçbir řey bilmez; ne yapılacaęı hep başka bir yerde yazılıdır.

Zengin model (rich model)

Kuralların, ilgili oldukları nesnelerin içinde bulunduęu yapı.

Örnek: Kapasitesi dolduęunda daha fazla yolcu almayı reddeden bir asansör. Kuralı asansörün kendisi uygular; dışarıda bekleyen bir görevliye ihtiyaç yoktur.

Davranıřın modele çekilmesi (behavior pull-up)

Dışarıda daęılmış kuralların, ait oldukları nesnelerin sorumluluęuna taşınması.

Örnek: Her řubede ayrı ayrı tutulan not defterlerindeki kuralların, ürünün üzerindeki kullanım etiketine taşınması. Kural artık ürünle birlikte gezer.

İlkel tip takıntısı (primitive obsession)

Anlamalı kavramların sade sayı veya metin olarak taşınması.

Örnek: Para tutarını yalnızca “250” diye yazmak. Lira mı, dolar mı belli deęildir. İki farklı para biriminin toplanması ancak iř iřten geçtikten sonra fark edilir.

Durum geçiři denetimi (state transition control)

Bir nesnenin hangi durumdan hangi duruma geçebileceęinin açıkça tanımlanması ve geçersiz geçiřlerin reddedilmesi.

Örnek: Teslim edilmiř bir kargonun tekrar “yola çıktı” durumuna geçememesi. Bu geçiři engelleyen, kargo kaydının kendisidir.

Dönüşüm Süreci

Olay tabanlı keşif (event storming)

İşin akışını, gerçekleşen olaylar üzerinden zaman sırasıyla yeniden kurma yöntemi.

Örnek: Bir kazanın nasıl olduğunu anlamak için olayları sırasıyla dizmek: “fren yapıldı”, “araç savruldu”, “çarpma gerçekleşti”. Sıralama tamamlandığında hangi noktada ne olduğu görünür hâle gelir.

Aşamalı dönüşüm (incremental transformation)

Yapının tek seferde değil, adım adım değiştirilmesi.

Örnek: Bir binanın kat kat tadilat edilmesi. Bina boşaltılmaz, hizmet kesilmez; her aşamada yaşanabilir durum korunur.

Bağlam bazlı ilerleme (context-by-context migration)

Dönüşümün önce tek bir alanda denenmesi, sonra diğerlerine yayılması.

Örnek: Yeni bir kasa sisteminin önce tek şubede denenmesi. Sorunlar bir şubede ortaya çıkar ve öğrenilenlerle diğer şubelere geçer.

Geçici çeviri noktası (temporary translation point)

Dönüşmüş bölüm ile henüz dönüşmemiş bölüm arasında kurulan, sonradan kaldırılacak ara bağlantı.

Örnek: Tadilat sırasında kurulan geçici merdiven. İşlevseldir, ama kalıcı olması amaçlanmaz; kalırsa binanın kusuru hâline gelir.

Geçiş göstergeleri (transition indicators)

Bir sonraki aşamaya geçme zamanının geldiğini bildiren gözlemlenebilir işaretler.

Örnek: Bir mağazada kuyrukların uzaması, yeni kasa açma zamanının geldiğini gösterir. Karar önseziyle değil, gözlemle verilir.

Erken soyutlama maliyeti (premature abstraction cost)

Henüz ihtiyaç doğmamışken kurulan yapının doğurduğu gereksiz yük.

Örnek: Üç kişilik bir aile için yirmi kişilik yemek düzeni kurmak. Kurulum zaman alır, bakımı sürekli ve hiçbir zaman karşılığı alınmaz.

Paketleme ve Dağıtım

Yerel dosya bağımlılığı (local file dependency)

Ortak bölümün, aynı makinedeki bir klasör üzerinden doğrudan kullanılması.

Örnek: Aynı evde oturan iki kişinin ortak alışveriş listesini mutfak dolabına asması. Ev içinde işler; başka eve taşınıldığında çalışmaz.

Paket deposu (package registry)

Ortak bölümün sürüm numarasıyla yayımlandığı merkezi kaynak.

Örnek: Bir yayınevinin kitap baskıları. Herkes “üçüncü baskı” der ve aynı içeriği kastettiğinden emin olur.

Tek depo (monorepo)

Tüm bölümlerin aynı kaynak deposunda tutulması.

Örnek: Bir ailenin tüm belgelerinin tek bir dolapta toplanması. Herhangi bir belge diğerleriyle birlikte, tek seferde güncellenebilir.

Sürüm sınırı (version boundary)

Ortak bölümdeki değişikliğin kullanıcılara ne zaman yansıtılacağını denetleyen eşik.

Örnek: Bir yönetmeliğin yürürlük tarihi. Metin bugün değişse bile, yürürlüğe girene kadar kimse ona göre davranmak zorunda değildir.

Bağlam başına paketleme (per-context packaging)

Her alanın ayrı bir paket olarak dağıtılması.

Örnek: Bir ansiklopedinin tek cilt yerine konu bazlı ciltler hâlinde basılması. İhtiyaç duyan yalnızca ilgili cildi edinir; bir cildin yeni baskısı diğerlerini etkilemez.

Birinci Kısım: Hazırlık Mimarisi

Üç Bileşenli Yapı

Önerilen başlangıç mimarisi üç ayrı sorumluluk alanından oluşur:

- **Çekirdek katman:** Domain modelleri, hesaplama ve karar fonksiyonları, dış servislerle iletişimi soyutlayan tanımlar ve doğrulama kuralları burada toplanır. Bu katman herhangi bir görsel arayüze veya çalışma platformuna bağımlı değildir.
- **Mobil istemci:** Kullanıcı arayüzünü ve cihaza özgü etkileşimleri üstlenir; iş kurallarını çekirdek katmandan alır.
- **Web istemci:** Tarayıcı ortamındaki arayüzü üstlenir; aynı çekirdek katmanı tüketir.

Bu ayrımın temel ilkesi şudur: *ne* yapılacağı çekirdek katmanda, *nasıl gösterileceği* istemci katmanlarda tanımlanır.

Çekirdek Katmanın Sınırları

Çekirdek katmanın kapsamına girmesi uygun olan unsurlar; veri tipleri ve domain nesneleri, saf hesaplama ve karar fonksiyonları, dış servis çağrılarını soyutlayan tanımlar ve doğrulama kurallarıdır. Buna karşılık arayüz bileşenlerine ait tanımlar, platforma özgü sistem nesneleri ve yalnızca tek bir platformda çalışabilen bağımlılıklar bu katmanın dışında bırakılmalıdır.

Bu sınırın korunmasının iki gerekçesi vardır. Yakın vadeli gerekçe, katmanın birden fazla istemci tarafından tüketilebilmesi ve bağımsız olarak test edilebilmesidir. Uzun vadeli gerekçe ise domain mantığının, arayüz teknolojilerinin yaşam döngüsünden bağımsız bir yerde birikmesidir; domain odaklı tasarımın gerektirdiği modelleme çalışması ancak böyle bir birikim üzerinde yürütülebilir.

Hazırlık Mimarisinin Sağladıkları ve Sağlamadıkları

Çekirdek katmanın ayrılması, domain odaklı tasarımın yapısal ön koşullarının bir bölümünü karşılar: domain mantığı arayüzden fiziksel olarak ayrılmış tek bir yerde toplanır, iş kuralları kullanıcı etkileşiminden bağımsız biçimde okunabilir hale gelir, model arayüz ortamı ayağa kaldırılmadan sınanabilir ve sınırlı bağlam ayrımı yapılabilecek fiziksel bir alan açılır.

Buna karşılık aşağıdaki koşullar bu ayrımdan türemez ve bağımsız olarak çalışılmayı gerektirir:

- **Ortak dil:** Domain terimlerinin ekip ile domain uzmanı arasında tek ve tutarlı bir anlamla kullanılması, bir klasör düzeninin sonucu değildir.

- **Bağlam sınırlarının isabeti:** Sınırlı bağlamların nerede başlayıp nerede biteceği, teknik bağımlılıklara değil işin gerçek ayırım çizgilerine göre belirlenir.
- **Model zenginliği:** Ayrılmış bir çekirdeğin içi tümüyle davranışsız veri nesneleri ve prosedürel işlevlerden oluşabilir; bu durumda düzenli bir kod tabanı vardır, ancak domain modeli yoktur.
- **Değişmezlerin sahipliği:** Hangi kuralın hangi nesne tarafından korunacağı ayrı bir modelleme kararıdır.

Bu nedenle hazırlık mimarisi, domain odaklı tasarımın ne yeterli ne de zorunlu koşuludur; geçişin maliyetini yeniden yazımdan yeniden düzenlemeye indiren bir ara aşamadır. Çalışmanın izleyen kısımları, yukarıda sıralanan karşılanmamış koşulların nasıl karşılanacağını konu edinmektedir.

İkinci Kısım: Ortak Dilin Kurulması

Ortak Dilin İşlevi

Ortak dil (ubiquitous language), domain uzmanı ile geliştirme ekibinin aynı kavramı aynı terimle ifade etmesi anlamına gelir. Bu, terminoloji birliğinden ibaret bir nezaket kuralı değil, modelin doğruluğunu denetleyen bir mekanizmadır. Kod içindeki bir adlandırma domain uzmanına anlamsız geliyorsa, o adlandırmanın arkasındaki model de büyük olasılıkla işin gerçekliğine karşılık gelmemektedir.

Ortak dilin en önemli özelliği, tek yönlü olmamasıdır: yalnızca işin dili koda taşınmaz, kodda ortaya çıkan ayrımlar da işin diline geri beslenir. Modelleme sırasında fark edilen bir ayrımın domain uzmanı tarafından da benimsenmesi, dilin gerçekten ortak hale geldiğinin göstergesidir.

Dilin Toplanma Yöntemi

Ortak dilin oluşturulması için izlenebilecek pratik yöntem şu adımlardan oluşur:

- Domain uzmanının işi kendi cümleleriyle anlattığı oturumların kayda alınması ve bu anlatımda geçen isimlerin, fiillerin ve durum adlarının ayıklanması.
- Ayıklanan terimlerin bir sözlükte tanımlanması; her terim için hangi durumlarda geçerli olduğunun ve hangi terimlerle karıştırılmaması gerektiğinin belirtilmesi.
- Terimlerin kod içindeki karşılıklarının bu sözlükle birebir eşleştirilmesi; sözlükte olmayan bir terimin koda girmemesi, kodda bulunan bir terimin sözlükte karşılığı yoksa ya sözlüğe eklenmesi ya da kodun yeniden adlandırılması.

Terim Çatışmalarının Ele Alınması

Aynı terimin işin farklı yerlerinde farklı anlamlar taşıması, ortak dil çalışmasının en sık karşılaşılan bulgusudur. Bu durum bir tutarsızlık değil, bir *keşiftir*: terimin anlamının değiştiği yer, çoğu zaman bir sınırlı bağlam sınırının geçtiği yerdir.

Çatışma karşısında izlenmesi gereken yol, terimi tek bir anlama zorlamak değildir. Böyle bir zorlama modeli işin gerçekliğinden uzaklaştırır ve sonuçta her iki tarafa da uymayan aşırı genel bir kavram üretir. Doğru yaklaşım, terimin her bağlamda kendi anlamıyla yaşamasına izin vermek ve bağlamlar arası geçişte açık bir çeviri tanımlamaktır.

Üçüncü Kısım: Sınırlı Bağlamların Keşfi

Sınırlı Bağlam Kavramı

Sınırlı bağlam (bounded context), belirli bir modelin ve o modele ait terimlerin geçerli olduğu alandır. Bir bağlamın içinde her terimin tek bir anlamı vardır; bağlamın dışında aynı terim geçerli olmayabilir veya başka bir anlam taşıyabilir.

Sınırlı bağlam, modülden farklıdır. Modül bir kod düzenleme birimidir; sınırlı bağlam ise bir *anlam* birimidir. İki modül aynı kavramı paylaşabilir, iki sınırlı bağlam ise tanımlı gereği paylaşamaz.

Sınırların Keşfinde Kullanılan Göstergeler

Bağlam sınırları tasarlanmaz, keşfedilir. Keşif için kullanılabilecek başlıca göstergeler şunlardır:

- **Dil kırılması:** Aynı terimin farklı anlamlarla kullanıldığı nokta, güçlü bir sınır adayıdır. Bu, en güvenilir göstergedir.
- **Değişim birlikteliği:** Birlikte değişme eğilimi gösteren kurallar aynı bağlama, birbirinden bağımsız değişkenler ayrı bağlamlara aittir. Bu ölçüt, geçmiş değişiklik kayıtları incelenerek gözlemsel olarak sınanabilir.
- **Sorumluluk ayrımı:** İşin farklı bölümlerinin farklı kişi veya birimler tarafından yürütülmesi, çoğu zaman bir bağlam ayrımına karşılık gelir.
- **Yaşam döngüsü farkı:** Bir kavramın farklı aşamalarda farklı verilere ve farklı kurallara tabi olması, o aşamaların ayrı bağlamlarda modellenmesini gerektirebilir.
- **Tutarlılık gereksinimi:** Anlık tutarlılık gerektiren kurallar aynı bağlam içinde kalmalı, gecikmeli tutarlılığın kabul edilebildiği yerler bağlam sınırı adayı olarak değerlendirilmelidir.

Keşif Çalışmasının Yürütülmesi

Sınır keşfi için yaygın olarak kullanılan yöntem, işin akışının olaylar üzerinden yeniden kurulmasıdır. Çalışma şu sırayla yürütülür:

- İşte gerçekleşen olayların, geçmiş zaman kipinde ve iş dilinde ifade edilerek zaman sırasına dizilmesi.
- Her olayı tetikleyen kararın ve o kararı veren rolün belirlenmesi.
- Kararın verilebilmesi için hangi bilgiye ihtiyaç duyulduğunun saptanması.
- Akış üzerinde dilin değiştiği, sorumluluğun el değiştirdiği veya beklemenin kabul edilebilir hale geldiği noktaların işaretlenmesi.
- İşaretlenen noktaların aday sınır olarak değerlendirilmesi ve her adayın yukarıdaki göstergeler karşısında sınanması.

Sınır Sayısına İlişkin Denge

Bağlam sayısının fazla tutulması, entegrasyon maliyetini ve çeviri yükünü artırır; az tutulması ise farklı anlamların tek bir modele sıkıştırılmasına ve modelin bulanıklaşmasına yol açar.

Pratik bir ölçüt, bir bağlamın tek bir ekip tarafından bütünüyle kavranabilir olmasıdır. Bir bağlamı anlatmak için sürekli istisna belirtmek gerekiyorsa bağlam fazla geniştir. Buna karşılık iki bağlam arasında sürekli veri gidip geliyor ve neredeyse her değişiklik ikisini birden etkiliyorsa, sınır yanlış yerden geçmektedir.

Dördüncü Kısım: Bağlam Haritası ve Bağlamlar Arası İlişkiler

Bağlam Haritasının İşlevi

Bağlam haritası (context map), sistemdeki sınırlı bağlamları ve aralarındaki ilişkileri gösteren üst düzey bir betimlemedir. Haritanın amacı yalnızca teknik bağımlılıkları göstermek değil, hangi bağlamın hangi bağlamın diline uymak zorunda olduğunu ve bu uyumun kim tarafından karşılanacağını açıkça ortaya koymaktır.

Her ilişkide bir *yukarı akış* ve bir *aşağı akış* taraf bulunur. Yukarı akış taraf, değişikliklerini kendi gereksinimlerine göre yapar; aşağı akış taraf ise bu değişikliklere uyum sağlamak zorundadır. Bu asimetrinin açıkça tanımlanmaması, bağlamlar arası bağımlılığın en sık görülen bozulma nedenidir.

İlişki Biçimleri

Bağlamlar arasında kurulabilecek başlıca ilişki biçimleri şunlardır:

- **Ortak çekirdek:** İki bağlamın modelin bir bölümünü paylaşmasıdır. Paylaşılan bölümde yapılacak her değişiklik iki tarafın da onayını gerektirir. Uyum maliyeti yüksek olduğundan yalnızca gerçekten ortak ve durağan kavramlar için tercih edilmelidir.
- **Müşteri–tedarikçi:** Aşağı akış tarafın ihtiyaçlarının, yukarı akış tarafın planlamasında dikkate alındığı ilişkidir. İki taraf arasında müzakere imkânı bulunduğu uygundur.
- **Uyum sağlayan:** Aşağı akış tarafın, yukarı akış modelini olduğu gibi benimsediği ilişkidir. Yukarı akış model kabul edilebilir nitelikteyse ve pazarlık gücü yoksa maliyeti en düşük seçenektir.
- **Yozlaşma önleyici katman:** Aşağı akış tarafın, dış modeli kendi diline çeviren bir ara katman kurmasıdır. Dış model kendi domain diline yabancı olduğunda veya kontrol dışı biçimde değiştiğinde tercih edilir. Maliyeti vardır, ancak dış modelin bozulmalarının kendi domainine sızmasını engeller.
- **Yayımlanmış dil:** Bağlamlar arası iletişimin, taraflardan hiçbirinin iç modeline ait olmayan, açıkça tanımlanmış ortak bir sözleşme üzerinden yürütülmesidir. Çok sayıda tüketicinin bulunduğu durumlarda uygundur.
- **Ayrı yollar:** İki bağlam arasında entegrasyon kurulmamasıdır. Entegrasyonun maliyeti sağlayacağı değeri aştığında meşru bir seçimdir.

Yozlaşma Önleyici Katmanın Konumu

Çekirdek–mobil–web yapısında yozlaşma önleyici katmanın en sık gerekli olduğu yer, çekirdek ile arka uç servisleri arasındaki sınırdır. Arka uçtan gelen veri temsili, çoğu zaman kalıcılık kaygılarıyla veya başka tüketicilerin ihtiyaçlarıyla biçimlenmiştir ve domain modelinin diline karşılık gelmez.

Bu katman, dış temsili domain nesnelere çeviren ve ters yönde domain kararlarını dış temsile dönüştüren bir sınır oluşturur. Katmanın varlığı, dış servis sözleşmesindeki bir değişikliğin etkisini tek bir çeviri noktasında hapseder; bu noktanın dışında domain modeli değişiklikten habersiz kalır.

Beşinci Kısım: Domain Modelinin Yapı Taşları

Kimlikli Nesneler ve Değer Nesneleri

Domain modelinin en temel ayrımı, bir kavramın kimliğiyle mi yoksa değeriyle mi tanımlandığıdır.

Kimlikli nesne (entity), süreklilik taşıyan ve nitelikleri değişse bile aynı kalan kavramdır. İki kimlikli nesnenin eşitliği, niteliklerinin değil kimliklerinin karşılaştırılmasıyla belirlenir. Bir kullanıcı, bir sipariş veya bir randevu bu niteliktedir.

Değer nesnesi (value object), yalnızca taşıdığı değerle tanımlanan, kimliği bulunmayan kavramdır. İki değer nesnesi, nitelikleri aynıysa eşittir. Bir para tutarı, bir tarih aralığı, bir adres veya bir ölçü birimi bu niteliktedir.

Modelleme pratiğinde en sık yapılan hata, değer nesnesi olması gereken kavramların ilkel tiplerle temsil edilmesidir. Bir para tutarının salt bir sayı olarak taşınması, para birimi uyumsuzluğu gibi hataların derleme zamanında değil çalışma zamanında ortaya çıkmasına yol açar. Değer nesnelerinin ayrı kavramlar olarak tanımlanması, ilgili kuralların dağılmak yerine tek bir yerde toplanmasını sağlar.

Değer nesnelerinin bir diğer önemli özelliği değişmez olmalarıdır: bir değer nesnesi oluşturulduktan sonra değiştirilmez, yeni bir değer gerektiğinde yenisi üretilir. Bu, paylaşılan nesnelerin beklenmedik biçimde değişmesinden kaynaklanan hata sınıfını tümüyle ortadan kaldırır.

Bütünlük Birimleri

Bütünlük birimi (aggregate), birlikte tutarlı kalması gereken nesnelerin oluşturduğu kümedir. Kümenin dışarıya açılan tek bir girişi bulunur; bu giriş *kök* olarak adlandırılır. Dışarıdan yapılan tüm değişiklikler kök üzerinden geçer ve kök, kümenin bütünlüğünü koruyan kuralların uygulanmasından sorumludur.

Bütünlük birimi kavramının varlık nedeni, değişmezlerin nerede korunacağı sorusuna kesin bir yanıt vermektir. Bir kural birden fazla nesneyi ilgilendiriyorsa ve her an geçerli olması gerekiyorsa, o kural bu nesneleri kapsayan bir bütünlük biriminin sorumluluğundadır.

Bütünlük Birimi Sınırının Çizilmesi

Bütünlük birimi sınırının belirlenmesi, domain odaklı tasarımın en zor kararıdır. Karar, aşağıdaki ölçütlere göre verilir:

- **Anlık tutarlılık gereksinimi:** Yalnızca aynı anda tutarlı olması *zorunlu* olan nesneler aynı birim içinde toplanmalıdır. Gecikmeli tutarlılığın kabul edilebildiği her yer, bir sınır adayıdır.
- **Değişiklik işlemi kapsamı:** Bir bütünlük birimi, tek bir işlemde bütünüyle değiştirilebilecek büyüklükte olmalıdır. Birden fazla birimi aynı işlemde değiştirme ihtiyacı, sınırların yanlış çizildiğinin göstergesidir.

- **Boyut:** Birim büyüdükçe yükleme maliyeti ve eşzamanlı erişim çakışması artar. Kural olarak birimler mümkün olan en küçük boyutta tutulmalıdır.
- **Referans biçimi:** Bir bütünlük birimi, başka bir birime nesne olarak değil yalnızca kimlik üzerinden referans vermelidir. Bu kural, birimlerin birbirine yapışmasını ve yükleme sınırlarının bulanıklaşmasını engeller.

Yaygın bir hata, veri modelindeki ilişkilerin doğrudan bütünlük birimi sınırı olarak benimsenmesidir. Veri modelindeki ilişki bir erişim kolaylığını, bütünlük birimi ise bir tutarlılık zorunluluğunu ifade eder; bu ikisi çoğu zaman örtüşmez.

Domain Olayları

Domain olayı (domain event), işin akışında anlam taşıyan ve gerçekleşmiş bir durumu bildiren kavramdır. Olaylar geçmiş zaman kipinde adlandırılır ve değişmezdir; gerçekleşmiş bir olay geri alınamaz, ancak telafi eden başka bir olayla dengelenebilir.

Domain olaylarının iki temel işlevi vardır. Birincisi, bütünlük birimleri arasındaki gecikmeli tutarlılığı mümkün kılmaktır: bir birimdeki değişiklik olay olarak yayımlanır, diğer birimler bu olaya kendi zamanlarında tepki verir. İkincisi, işin akışını kod içinde açıkça görünür kılmaktır; olay adları, ortak dilin en doğrudan kod karşılığıdır.

Domain Servisleri

Bazı iş kuralları, hiçbir kimlikli nesneye veya değer nesnesine doğal olarak ait değildir; birden fazla nesneyi ilgilendirir ve herhangi birinin sorumluluğuna zorlandığında model yapaylaşır. Bu tür kurallar **domain servisi** olarak modellenir.

Domain servisi, kendi başına durum taşımayan ve yalnızca domain kavramlarıyla çalışan bir işlem birimidir. Adlandırması, teknik bir işlem değil bir iş eylemi olmalıdır. Domain servislerinin sayıca artması, modelin anemikleşmeye başladığının erken göstergesidir: davranışın nesnelerin içine çekilemediği ve dışarıda biriktiği anlamına gelir.

Depolar

Depo (repository), bütünlük birimlerinin saklanması ve geri getirilmesi işlemini domain diliyle ifade eden soyutlamadır. Depo, bir veri erişim aracının ince bir sarmalayıcısı değildir; arayüzü, veri erişim işlemleriyle değil domain ihtiyaçlarıyla tanımlanır.

Depoların tasarımında üç ilke geçerlidir:

- Her bütünlük birimi için tek bir depo tanımlanır; birim içindeki alt nesneler için ayrı depo oluşturulmaz.
- Depo arayüzü domain katmanında, gerçekleştirimi ise dış katmanda konumlandırılır. Böylece domain, kalıcılık teknolojisinden habersiz kalır.

- Depo, bütünlük birimini bütün olarak döndürür; kısmi yükleme, birimin bütünlük güvencesini zedeler.

Üretim Sorumluluğu

Bir bütünlük biriminin oluşturulması, çoğu zaman basit bir değer ataması değildir; oluşturma anında sağlanması gereken kurallar bulunur. Bu kurallar oluşturmaya yapan tarafa bırakıldığında, her çağrı noktasında tekrarlanmak zorunda kalır ve zamanla birbirinden ayrışır.

Bu nedenle oluşturma sorumluluğu, kuralları tek bir yerde uygulayan bir üretim noktasında toplanır. Üretim noktası, geçerli olmayan bir nesnenin hiçbir zaman var olamamasını güvence altına alır; bu, model bütünlüğünün en ucuz koruma biçimidir.

Altıncı Kısım: Katmanların Ayrıştırılması

Domain ve Uygulama Katmanları

Domain odaklı bir çekirdek, iki ayrı iç katmandan oluşur.

Domain katmanı, iş kavramlarını ve kurallarını barındırır. Bu katman hiçbir dış bileşene bağımlı değildir; ne veri tabanını, ne dış servisleri, ne de kendisini kimin çağırdığını bilir. Bağımsızlığı, modelin işin gerçekliğine göre şekillenmesini ve teknik kaygılarla bozulmamasını sağlar.

Uygulama katmanı, kullanım senaryolarını yürütür. Bir senaryonun akışını düzenler: gerekli bütünlük birimlerini depolardan alır, üzerlerinde domain işlemlerini çağırır, sonucu depoya geri yazar, işlem sınırını yönetir ve ortaya çıkan olayların yayımını sağlar. Bu katmanda iş kuralı bulunmaz; yalnızca kuralların hangi sırayla uygulanacağı tanımlanır.

Ayrımın sınanması için kullanılacak ölçüt şudur: uygulama katmanındaki bir işlem, kendisi karar veriyorsa yanlış yerrdedir. Karar domain katmanına, kararın hangi koşullarda tetikleneceği uygulama katmanına aittir.

Bağımlılık Yönü

Domain odaklı mimarinin taşıyıcı ilkesi, bağımlılıkların içe doğru akmasıdır. Dış katmanlar iç katmanları bilir; iç katmanlar dış katmanlardan habersizdir.

Bu ilke, dış dünyaya erişim gerektiren durumlarda arayüz tanımının içeride, gerçekleştiriminin dışarıda konumlandırılmasıyla korunur. Domain katmanı bir veri kaynağına veya dış servise ihtiyaç duyduğunda, ihtiyacını kendi diliyle bir arayüz olarak tanımlar; bu arayüzün nasıl karşılanacağı dış katmanların sorunudur.

İstemcilerin Konumu

Domain odaklı yeniden yapılandırma sonrasında mobil ve web istemcileri, çekirdeğin domain nesnelerine doğrudan erişmez. İstemcilerin gördüğü yüzey, uygulama katmanının kullanım senaryolarıdır.

Bu kısıtın gerekçesi, domain nesnelerinin istemcilere açılması hâlinde arayüz ihtiyaçlarının model üzerinde baskı kurmasıdır. Bir ekranın ihtiyaç duyduğu veri biçimi modele yansıtıldığında, model işin değil arayüzün diline göre şekillenmeye başlar.

İstemcilere döndürülen veriler bu nedenle domain nesneleri değil, yalnızca aktarım amacı taşıyan sade yapılar olmalıdır. Bu yapılar arayüz ihtiyaçlarına göre serbestçe biçimlenebilir; model üzerinde hiçbir etkileri olmaz.

Yedinci Kısım: Kalıcılık ve Model Arasındaki Gerilim

Gerilimin Kaynağı

Bir sistemde iki ayrı biçimlendirme kaygısı yan yana yaşar. Domain modeli işin kavramlarına göre biçimlenir: neyin bir bütün sayıldığı, hangi kuralın nerede yaşadığı, bir kavramın hangi koşulda geçerli olduğu. Veri şeması ise saklama ve sorgulama verimliliğine göre biçimlenir: kaç tablo, hangi kolon, hangi dizin, hangi birleştirme maliyeti.

Bu iki kaygı çoğu zaman örtüşmez. Domainde bölünmez sayılan bir kavram şemada iki kolona ayrılır; domainde bir bütünün parçası olan bir liste şemada ayrı bir tabloya taşınır. Örtüşmemenin kendisi bir kusur değildir, çünkü iki tarafın da kendi gerekçesi vardır. Kusur, örtüşmediği hâlde birinin diğerine zorla uydurulmasıdır. Pratikte geçişi en sık tıkayan nokta da tam olarak burasıdır.

Gerilimin çözümü taraflardan birini diğerine feda etmek değil, aralarındaki dönüşümü açık bir sorumluluk hâline getirmektir. Dönüşüm zaten her durumda yapılmaktadır; soru yalnızca onun görünür bir yerde mi yoksa kod boyunca dağılmış hâlde mi yapıldığıdır. Bu dönüşüm depo gerçekleştirmenin işidir ve domain katmanını dönüşümden habersiz kalmalıdır.

Şema Baskısına Karşı Korunma

Baskının Göstergeleri

Dönüşüm sorumluluğu bir yere yerleştirilmediğinde veri şeması modelin içine doğru genişler. Bu genişlemenin gözlenebilir üç göstergesi vardır:

- Domain nesnelerinin yalnızca kalıcılık aracının gerektirdiği için taşıdığı alanlar bulunması. Sürüm numaraları, silinme damgaları, eşleme etiketleri gibi işin dilinde karşılığı olmayan öğeler kavramın parçasıymış gibi görünmeye başlar.
- Nesne yapısının tablo yapısını birebir yansıtması. Domainde bütün olan kavramlar tablo kolonlarına ayrıştığı için, o kavrama ait kuralın dayatılacağı bir yer kalmaz; kural nesnenin dışına, çağrı yerlerine dağılır.
- Kavramların davranışsız veri torbalarına dönüşmesi. Alanlar dışa açılır, kurallar servis katmanına taşınır ve nesne bir kavram olmaktan çıkıp taşıma kabına indirgenir.

Ayrımın Kurulması

Bu baskıya karşı korunma, domain nesneleri ile kalıcılık temsillerinin ayrı tutulmasıyla sağlanır. Kalıcılık temsili tabloyu bilinçli olarak yansıtır ve yansıtması beklenir; domain nesnesi ise işin kavramını yansıtır. İkisi arasındaki eşleme deponun içinde, tek bir yerde yapılır.

Ayrım bir dönüşüm maliyeti doğurur. Bu maliyet elle yazılan eşleme kodudur ve küçük ölçekte gereksiz görünür. Ancak maliyetin karşılığı, modelin işin diliyle konuşmaya devam etmesidir: şema değiştiğinde domain katmanı sabit kalır, model yeniden düzenlendiğinde şema zorlanmaz. Bu nedenle ayrımın geri dönüşü karmaşıklık arttıkça artar.

Okuma ve Yazma Yollarının Ayrılması

İlkenin Kapsamı

Bütünlük birimi ilkeleri yazma tarafı için tasarlanmıştır. Birimin bütün olarak yüklenmesi, kurallarının uygulanması ve bütün olarak kaydedilmesi, veri değiştiren işlemler için doğru davranıştır; çünkü bir kuralın doğrulanabilmesi için birimin tam durumuna ihtiyaç vardır.

Bu ilkenin okuma amaçlı işlemlere de genişletilmesi, kapsamının yanlış anlaşılmasından doğar. Yalnızca ekranda gösterilecek bir liste için bütünlük birimlerinin tam olarak yüklenmesi ve ardından sadeleştirilmesi, ne bir kural uygular ne de bir tutarlılık sağlar. Yapılan tek şey, kullanılmayacak durumun kurulup atılmasıdır.

Ayrımın Biçimi

Bu nedenle okuma yolunun yazma yolundan ayrılması meşrudur. Sorgulama işlemleri doğrudan veri kaynağından, ekranın ihtiyaç duyduğu sade yapıları üretmek yürütülebilir; bu yolda domain nesnesi hiç kurulmaz. Böylece iki yol farklı gerekçelerle ve farklı biçimlerde optimize edilir: yazma yolu doğruluk için, okuma yolu verimlilik için.

Ayrımın Koşulu

Ayrımın koşulu açıktır: okuma yolu hiçbir durum değişikliği yapmaz ve hiçbir iş kuralı uygulamaz.

Bu koşul bir biçimsel gereklilik değil, ayrımı meşru kılan şeyin kendisidir. Okuma yolu bir iş kuralı uygulamaya başlarsa, o kural artık iki yerde yaşar; iki kopya zamanla ayrışır ve hangisinin bağlayıcı olduğu belirsizleşir. Okuma yolu durum değiştirirse, veri kuralı uygulayan mekanizmadan geçmeden değişmiş olur ve bütünlük biriminin sağladığı güvence tümüyle ortadan kalkar.

Koşul korunduğu sürece okuma yolu bir izdüşümden ibarettir: veriyi ekranın istediği biçimde sunar, hakkında karar vermez. Karar verme yetkisi bütünüyle yazma yolunda kaldığı için model bütünlüğü zedelenmez.

Sekizinci Kısım: Anemik Modelden Zengin Modele Geçiş

Anemik Modelin Teşhisi

Yapının Tanımı

Anemik model, veri taşıyan nesnelerin davranış içermediği ve tüm kuralların dışarıdaki işlem birimlerinde toplandığı yapıdır. Bu yapıda nesne bir kavramı değil, bir taşıma kabını temsil eder; kavrama ait kurallar ise nesneyi kullanan çağrı yollarına dağılmıştır.

Yapının belirleyici özelliği, kuralın *veriye* değil *yola* bağlanmış olmasıdır. Bir kural yalnızca kendisini içeren işlem biriminden geçildiğinde uygulanır; aynı veriye başka bir yoldan erişen her yeni kod, kuralı yeniden hatırlamak ve yeniden yazmak zorundadır. Anemik modelin ürettiği hasarın kaynağı budur: kural ihlal edilmez, yalnızca atlanır.

Göstergeler

Yapının varlığı şu göstergelerden anlaşılır:

- Domain nesnelerinin yalnızca alan okuma ve yazma işlemlerinden oluşması. Nesne, işin dilinde bir eylem sunmaz; yalnızca içeriğinin dışarıdan alınmasına ve değiştirilmesine izin verir.
- Kuralların, nesneleri parametre olarak alan dış işlevlerde bulunması. Bir işlevin tek parametresi belirli bir nesne ise, o kural gerçekte o nesneye aittir ve dışarıda tutulması bir tasarım tercihidir.
- Aynı doğrulamanın birden fazla çağrı noktasında tekrarlanması. Tekrar, kuralın ortak bir sahibi olmadığının doğrudan kanıtıdır; kopyalar zamanla ayrışır ve hangisinin bağlayıcı olduğu belirsizleşir.
- Bir nesnenin geçersiz bir duruma girmesinin mümkün olması ve bu durumun ancak sonradan denetlenmesi. Geçersiz nesne, oluşturulduğu an ile denetlendiği an arasında sistemde serbestçe dolaşır.

Davranışın Modele Çekilmesi

Anemik yapıdan zengin modele geçiş tek seferde değil, kademeli olarak yürütülür. Aşağıdaki adımlar bir sıra önerir; her adım bir öncekinin açtığı alanı kullanır.

Kuralların Toplanması

Dışarıdaki işlevler taranarak, tek bir nesnenin verilerine dayanan kurallar tespit edilir ve o nesnenin sorumluluğuna taşınır. Ölçüt basittir: kuralın doğrulanması için yalnızca o nesnenin kendi verisi yeterliyse, kuralın yeri o nesnedir.

Bu adım henüz yapıyı değiştirmez; yalnızca dağınık kuralların adreslerini belirler ve sonraki adımların üzerinde çalışacağı listeyi oluşturur.

Değer Nesnelerinin Çıkarılması

Birlikte anlam taşıyan ve birlikte doğrulanan alan kümeleri ayrı değer nesneleri olarak tanımlanır. Ayrı ayrı ele alındığında anlamsız olan, yalnızca bir arada geçerlilik kazanan alanlar, gerçekte tek bir kavramın parçalanmış hâlidir.

İlgili doğrulama kuralları bu nesnelerin oluşturulmasına bağlanır. Bağlamanın sonucu belirleyicidir: kural artık uygulanması hatırlanan bir adım değil, kavramın var olma koşuludur. Geçersiz bir değer nesnesi üretilmediği için, onu kullanan hiçbir kodun ayrıca denetim yapması gerekmez.

Doğrudan Alan Yazımının Kapatılması

Nesnelerin alanlarının dışarıdan serbestçe değiştirilmesi engellenir; değişiklikler yalnızca iş eylemi adı taşıyan işlemler üzerinden yapılabilir hâle getirilir.

Adın taşıdığı fark önemlidir. Alan atama biçimindeki bir işlem mekaniktir, hiçbir ön koşul sormaz ve kendisine verilen her değeri kabul eder. İşin dilinden alınmış bir eylem adı ise bir olayı temsil eder ve o olayın gerçekleşmesi için gereken koşulları kendi içinde taşıyabilir. Böylece kural, değişikliğin geçtiği yolun üzerine değil, değişikliğin kendisine yerleşir.

Durum Geçişlerinin Denetlenmesi

Bir nesnenin hangi durumdan hangi duruma geçebileceği açıkça tanımlanır ve geçersiz geçişler nesnenin kendisi tarafından reddedilir.

Durum geçişleri, anemik yapıda en sık dışarıda bırakılan kural türüdür; çünkü her çağrı noktası kendi bağlamında hangi geçişin uygun olduğunu bildiğini varsayar. Geçiş kümesinin nesnenin içinde tanımlanması, bu varsayımı gereksiz kılar: hangi yoldan gelinirse gelinsin, tanımsız geçiş kabul edilmez.

Artakalanın Değerlendirilmesi

Yukarıdaki adımlardan sonra dışarıda kalan kurallar incelenir. Üç durum ayırt edilir:

- Birden fazla nesneyi ilgilendiren kurallar domain servisi olarak bırakılır. Bu kuralların tek bir doğal sahibi yoktur; herhangi birine yüklenmeleri yapay bir bağımlılık üretir.
- Yetkilendirme, kayıt tutma, işlem sınırı gibi uygulama düzeyindeki kaygılar zaten modele ait değildir ve dışarıda kalmaları doğrudur.
- Bu iki gruba girmediği hâlde yerleştirilemeyen kurallar bir modelleme sinyalıdır. Böyle bir kural genellikle henüz adı konmamış bir kavrama işaret eder; bu durumda yapılması gereken kuralı zorla bir yere yerleştirmek değil, modeli yeniden gözden geçirmektir.

Zengin Modelin Sınırı

Her nesnenin davranışla doldurulması bir amaç değildir. Bazı kavramlar gerçekten de yalnızca veri taşır: dış dünyaya sunulan temsiller, okuma yolunun ürettiği sade yapılar, yapılandırma nesneleri. Bunları zorla davranışlandırmak modeli zenginleştirmez, yalnızca yapaylık üretir ve gerçek kuralların bulunduğu yerleri gölgeler.

Ölçüt, kuralın *doğal sahibinin* bulunup bulunmadığıdır. Kural bir nesnenin kendi verileriyle ifade edilebiliyorsa o nesneye aittir; edilemiyorsa dışarıda kalması doğrudur. Bu ölçüt iki yönlü çalışır: hem kuralın modele çekilmesini gerekçelendirir, hem de çekilmemesi gereken kuralların dışarıda bırakılmasını meşrulaştırır.

Yedinci Kısım'da modelin kalıcılık tarafından biçimlendirilmesine karşı korunması ele alınmıştı. Bu kısımda ele alınan ise modelin uygulama katmanı tarafından boşaltılmasına karşı korunmasıdır. İki durum farklı yönlerden gelen aynı sorunu gösterir: modelin kendi sınırını ve kendi sorumluluğunu koruyamaması.

Dokuzuncu Kısım: Çekirdek Katmanın Projeler Arasında Paylaşımı

Paylaşım Yöntemleri

Çekirdek katmanın istemci projelere ulaştırılması için başlıca dört yöntem bulunur:

- **Yerel dosya bağımlılığı:** Tek geliştiricili ve tek makinelı durumlarda hızlıdır; dizin yapısının her makinede aynı olmasını varsaydığı için ekip ortamında sürdürülemez.
- **Paket deposu üzerinden yayın:** Çok geliştiricili ve uzun ömürlü projelerde en sağlam çözümdür. Tüm geliştiriciler ve otomatik derleme süreçleri paketi aynı kaynaktan alır.
- **Sürüm kontrol deposu üzerinden bağımlılık:** Kurulum maliyeti düşüktür; sürüm yönetimi dal veya işlem referanslarına dayandığı için daha dağınık kalır.
- **Tek depo yaklaşımı:** Dizin hiyerarşisini tüm geliştiriciler için aynı kılar ve çekirdek ile istemciler arasındaki değişikliklerin tek bir işlem akışında yönetilmesine olanak tanır.

Yöntem Seçiminin Domain Odaklı Tasarımla İlişkisi

Paylaşım yöntemi, model olgunluğuna göre değerlendirilmelidir.

Model henüz oturmamışken sık yineleme yapılır ve her yineleme istemcileri de etkiler. Bu dönemde tek depo yaklaşımı, değişikliğin çekirdek ve istemcilerde eşzamanlı yapılabilmesini sağlayarak yineleme maliyetini en aza indirir.

Model kararlı hale geldikten sonra ise paket deposu üzerinden yayın öne çıkar. Sürüm sınırı, domain modelinde yapılan değişikliklerin istemcilere ne zaman ve nasıl yansıtılacağını denetim altına alır; bu, uygulama katmanı yüzeyinin bir sözleşme olarak ele alınmasını mümkün kılar.

Bağlam Başına Paketleme

Bağlam sayısı arttığında, çekirdeğin tek bir paket olarak dağıtılması sorgulanmaya başlar. Her sınırlı bağlamın ayrı paket olarak yayımlanması, istemcilerin yalnızca ihtiyaç duydukları bağlamlara bağımlı olmasını sağlar ve bağlamların birbirinden bağımsız sürümlenmesine imkân verir.

Buna karşılık paket sayısının artması sürüm eşgüdüm maliyetini yükseltir. Ayrıştırma kararı, bağlamların gerçekten bağımsız evrilip evrilmediğine bakılarak verilmelidir; birlikte değişen bağlamların ayrı paketlere bölünmesi yalnızca ek yük üretir.

Onuncu Kısım: Evrimsel Yol Haritası

Birinci Aşama: Çalışan İlk Sürüm

Öncelik çalışan bir ürün ortaya koymaktır. Bu aşamada katı mimari kalıplar zorunlu değildir; ancak arayüz kodu ile iş mantığının kaba düzeyde ayrı alanlarda tutulması, ileride yapılacak yeniden düzenlemeyi belirgin ölçüde kolaylaştırır.

İkinci Aşama: İş Mantığının Netleşmesi

Ürün temel işlevlerini kazandıktan sonra, aynı kuralın birden çok ekranda tekrarlandığı noktalar görünür hale gelir. Bu aşamada saf iş fonksiyonları ayrı bir alanda toplanır ve ekranlar, bu fonksiyonları çağırıp sonucu gösteren daha ince katmanlara dönüşür.

Toplanan işlevler bu noktada işlevsel benzerliğe göre değil, ait oldukları iş alanına göre gruplanmalıdır. Arayüz temelli gruplama ileride bağlam sınırlarını gizlerken, iş alanı temelli gruplama bu sınırların ilk taslağını oluşturur.

Bu aşamada başlatılması gereken bir diğer çalışma, terim toplamadır. Domain uzmanıyla yürütülen konuşmalarda geçen terimlerin kayda alınması, ortak dil çalışmasının hiçbir maliyet doğurmayan başlangıcıdır ve ileride bağlam keşfinin ham verisini oluşturur.

Üçüncü Aşama: Çekirdeğin Ayrılması ve Web İstemcisinin Eklenmesi

Web istemcisi ihtiyacı doğduğunda, önceki aşamada derlenmiş olan iş fonksiyonları bağımsız bir çekirdek pakete taşınır. Mobil proje bu fonksiyonları artık yerel dizinden değil paketten alır; yeni oluşturulan web projesi de aynı paketi bağımlılık olarak ekler.

Bu aşamanın kritik yan ürünü, çekirdek katmanının dış yüzeyinin ilk kez açıkça tanımlanmış olmasıdır. İstemcilerin hangi işlevlere eriştiği görünür hale geldiğinde, bu yüzey ileride uygulama katmanının kullanım senaryolarına dönüşecek çağrı kümesini oluşturur.

Dördüncü Aşama: Bağlam Keşfi

Bu aşamada henüz kod taşınmaz; yalnızca haritalama yapılır. Yürütülecek çalışmalar şunlardır:

- İş akışının olaylar üzerinden yeniden kurulması ve karar noktalarının belirlenmesi.
- Toplanan terimlerin sözlükleştirilmesi ve anlam çatışmalarının işaretlenmesi.
- Geçmiş değişiklik kayıtlarının incelenerek birlikte değişen kural kümelerinin tespiti.

- Aday sınırların belirlenmesi ve her adayın dil kırılması, değişim birlikteliği, sorumluluk ayrımı ve tutarlılık gereksinimi ölçütleri karşısında sınanması.
- Aday bağlamlar arasındaki ilişkilerin ve her ilişkide yukarı/aşağı akış tarafların belirlenmesi.

Aşamanın çıktısı çalışan kod değil, bir bağlam haritası ve terim sözlüğüdür. Bu çıktının kod yazılmadan önce üretilmesi, yanlış sınırla yapılan taşımanın maliyetinden kaçınmayı sağlar.

Beşinci Aşama: İlk Bağlamanın Yapılandırılması

Tüm çekirdeğin aynı anda dönüştürülmesi yüksek risklidir. Bunun yerine tek bir bağlam seçilerek dönüşüm orada uygulanır ve yaklaşım sınanır.

Seçim ölçütü, bağlamanın hem yeterince değerli hem de yeterince yalıtık olmasıdır: iş açısından merkezi ama diğer alanlarla bağı görece zayıf olan bağlam idealdir. En karmaşık bağlamla başlamak öğrenme maliyetini riske dönüştürür; en önemsiziyle başlamak ise dönüşümün değerini görünmez kılar.

Seçilen bağlamda yürütülecek işlemler sırasıyla şunlardır: kimlikli nesne ve değer nesnesi ayrımının yapılması, bütünlük birimi sınırlarının çizilmesi, depoların tanımlanması, kullanım senaryolarının uygulama katmanına yerleştirilmesi ve davranışın modele çekilmesi.

Altıncı Aşama: Sınırların Sağlamlaştırılması

İlk bağlam yapılandırıldığında, diğer alanlarla arasındaki bağlantılar açıkta kalır. Bu aşamada söz konusu bağlantılar açık ilişkilere dönüştürülür.

Dış servislerle olan bağlantılar için yozlaşma önleyici katman kurulur; henüz dönüştürülmemiş alanlarla olan bağlantılar için geçici çeviri noktaları tanımlanır. Bu geçici noktalar, dönüşümün geri kalanı tamamlandıkça kaldırılacak iskele niteliğindedir; kalıcı yapı olarak bırakılmaları teknik borç üretir.

Yedinci Aşama: Yaygınlaştırma

İlk bağlamdan elde edilen deneyimle, kalan bağlamlar tek tek dönüştürülür. Her dönüşüm sonrasında bağlam haritası güncellenir ve ilişki biçimlerinin hâlâ geçerli olup olmadığı gözden geçirilir.

Bu aşamada sık karşılaşılan bir bulgu, keşif aşamasında çizilen bazı sınırların yanlış yerden geçtiğinin anlaşılmasıdır. Bu bir başarısızlık değil, modelin olgunlaşmasının olağan seyridir. Sınırların yeniden çizilmesi, dönüşüm bağlam bazında yürütüldüğü için sınırlı bir maliyetle mümkün olur.

Sekizinci Aşama: Süreklilik

Domain odaklı tasarım bir varış noktası değil, sürdürülen bir uygulamadır. Süreklilik için gereken pratikler şunlardır:

- Ortak dilin canlı tutulması: yeni terimlerin sözlüğe eklenmesi, kullanımdan düşen terimlerin çıkarılması.
- Bağlam haritasının güncel tutulması ve yeni bağımlılıkların haritaya işlenmesi.
- Bütünlük birimi sınırlarının, işlem çakışmaları ve performans sorunları ışığında düzenli olarak gözden geçirilmesi.
- Domain servislerinin sayısının izlenmesi; artış eğilimi, modelin yeniden anemikleşmeye başladığının göstergesidir.

Aşama Geçişlerine İlişkin Göstergeler

Her aşamaya geçişin erken yapılması karşılığı olmayan soyutlama maliyeti, geç yapılması ise birikmiş karmaşıklık doğurur. Geçiş zamanının geldiğine dair göstergeler şunlardır:

- **Dördüncü aşamaya (bağlam keşfi):** Aynı terimin farklı alanlarda farklı anlamlarla kullanılmaya başlanması; bir kuralda yapılan değişikliğin ilgisiz görünen yerlerde düzeltme gerektirmesi; yeni bir işlevin nereye konulacağına tekrar tekrar tartışılması.
- **Beşinci aşamaya (ilk bağlam):** Bağlam haritasının üretilmiş ve ekip içinde üzerinde uzlaşmış olması; en az bir bağlamın sınırlarının istikrarlı biçimde tarif edilebilmesi.
- **Yedinci aşamaya (yaygınlaştırma):** İlk bağlamda en az bir tam özellik döngüsünün yeni yapıyla tamamlanmış olması ve yapının değişiklik hızını yavaşlatmadığının gözlenmesi.

Bu göstergelerin hiçbiri gözlenmiyorsa mevcut yapı yeterlidir ve geçiş ertelenmelidir.

On Birinci Kısım: Yaygın Hatalar

Dönüşüm sırasında sık karşılaşılan hatalar ve bunların teşhis göstergeleri aşağıda özetlenmektedir:

- **Klasör düzenini dönüşüm sanmak:** Dizin adları değişmiş, ancak kurallar hâlâ dışarıdaki işlevlerde durmaktadır. Teşhis: domain nesnelerinde davranış bulunmaması.
- **Bağlam sınırlarını teknik ölçütlere göre çizmek:** Sınırlar veri tablolarına veya arayüz ekranlarına göre belirlenmiştir. Teşhis: aynı iş kavramının birden çok bağlamda parçalı biçimde bulunması.
- **Bütünlük birimini fazla geniş tutmak:** Neredeyse tüm veri tek bir kök altında toplanmıştır. Teşhis: her işlemde büyük nesne kümelerinin yüklenmesi ve eşzamanlı erişim çakışmaları.
- **Bütünlük birimleri arasında nesne referansı kullanmak:** Birimler birbirine nesne düzeyinde bağlanmıştır. Teşhis: yükleme sınırının belirsizleşmesi ve tek bir işlemde birden çok birimin değişmesi.
- **Uygulama katmanına kural sızdırmak:** Kullanım senaryoları koşul denetimleri içermeye başlamıştır. Teşhis: aynı denetimin birden fazla senaryoda tekrarlanması.
- **Domain nesnelerini istemcilere açmak:** Arayüz ihtiyaçları model üzerinde baskı kurmaktadır. Teşhis: domain nesnelerinde yalnızca gösterim amacı taşıyan alanların belirmesi.
- **Her yere yozlaşma önleyici katman koymak:** Kendi kontrolündeki bileşenler için de çeviri katmanı kurulmuştur. Teşhis: hiçbir kural içermeyen, yalnızca alan eşleyen katmanların çoğalması.
- **Tüm sistemi aynı anda dönüştürmeye çalışmak:** Dönüşüm bağlam bazında değil bütünsel yürütülmektedir. Teşhis: uzun süre teslim edilebilir çıktı üretilmemesi.

Sonuç

Bu çalışma, çok istemcili bir uygulamada iş mantığının merkezileştirilmesinden başlayarak domain odaklı bir mimariye ulaşan bütünsel bir hat sunmaktadır.

Hattın ilk bölümü, çekirdek katmanın arayüzden ayrılmasıdır. Bu ayrım kendi başına domain odaklı tasarım değildir; sağladığı şey, modelleme çalışmasının yürütülebileceği bir zemin ve geçiş maliyetinin yeniden yazımdan yeniden düzenlemeye inmesidir.

İkinci bölüm, modelleme çalışmasının kendisidir: ortak dilin kurulması, sınırlı bağlamların dil kırılması ve değişim birlikteliği gibi gözlemsel göstergelerle keşfedilmesi, bağlamlar arası ilişkilerin açık biçimde tanımlanması, model yapı taşlarının seçilmesi ve davranışın nesnelere çekilmesi. Bu çalışmanın hiçbir bölümü dizin düzeninden türemez; her biri ayrı bir karar gerektirir.

Üçüncü bölüm, kalıcılık ile model arasındaki gerilimin yönetilmesidir. Bu gerilim geçişin pratikte en sık tıkanıdığı yerdir ve çözümü, dönüşümü depo gerçekleştiriminde yoğunlaştırarak domain katmanını şema baskısından yalıtımdır.

Dönüşümün yürütülmesinde iki ilke belirleyicidir. Birincisi, aşamalılık: her mimari yatırım ancak karşılık gelen ihtiyaç gözlemlendiğinde yapılmalıdır. İkincisi, bağlam bazlı ilerleme: dönüşüm tek bir bağlamda sınanmalı, elde edilen deneyimle yaygınlaştırılmalıdır. Bu iki ilke birlikte, hem erken soyutlama maliyetinden hem de bütünsel dönüşümün riskinden korunmayı sağlar.

Son olarak, domain odaklı tasarımın bir varış noktası olmadığı vurgulanmalıdır. Ortak dil canlı tutulmadığında, bağlam haritası güncellenmediğinde ve bütünlük birimi sınırları gözden geçirilmediğinde, en özenle kurulmuş model dahi zamanla anemikleşir. Yapının değeri, kurulduğu andan değil sürdürüldüğü süreden gelir.